File Transfer and Access (FTP, TFTP, NFS)

Reti di Calcolatori (Modulo 2)

Introduction

- Network systems allow access to remote files

- Variety of approaches to access files on remote machines
  - Centralized file server provides storage for diskless machines (lower costs)
  - Remote storage to archive data (backup)
  - Share data across multiple programs, users, sites

On-Line File Sharing

- Always a popular application

- Two basic paradigms
  - Piecewise file access (online shared access)
    - Multiple programs access the same file concurrently
    - Changes are available to all programs immediately
  - Whole-file copying
    - The program obtains a local copy of the shared file
    - The program makes a change and copies back the file

On-Line File Sharing

- Piecewise access mechanism
  - Sophisticated and easy to use → Gestisce concorrenza automatica
  - *Opaque*: application uses special facilities to access remote file
  - *Transparent*: application uses same facilities to access local and remote files
    - Remote files integrated with local files

On-Line File Sharing

- Advantages
  - No change to the applications
  - No need of special clients

- Disadvantages
  - Unreliable connections make application unreliable (slow, not usable…)
  - Transparent access difficult to implement
    - Difficult to map file names in one file system in another
    - Must handle ownership, authorization, access protection
    - Difficult to implement all operations on files

File Transfer

- Whole file copying
- Client
  - Contacts server
  - Specifies file
  - Specifies transfer direction
- Server
  - Maintains set of files on local disk
  - Waits for requests
  - Honors request from client

File Transfer

- 2-steps access to data
  - The user obtains a local copy
  - The user operates on the copy
- A user uses a special purpose program for file transfer
  - Specify the remote pc and the file to transfer, and the username/password to access them
- Application programs are used locally
  - More efficient
  - More robust to network malfunctioning

File Transfer

- Transfer between heterogeneous machines difficult
  - Client and server must agree on authorization ownership, access protection
  - Data formats may be not compatible
    - Inverse translations not possible
    - E.g., two representation for floating point; impossible to translate without losing precision

File Transfer Protocol (FTP)

- Originates from file transfer protocol in ARPANET
- Major TCP/IP protocol for whole-file copying
- Uses TCP for transport
- Major Internet application until the WWW/email era

FTP: Features

In addition to file transfer function

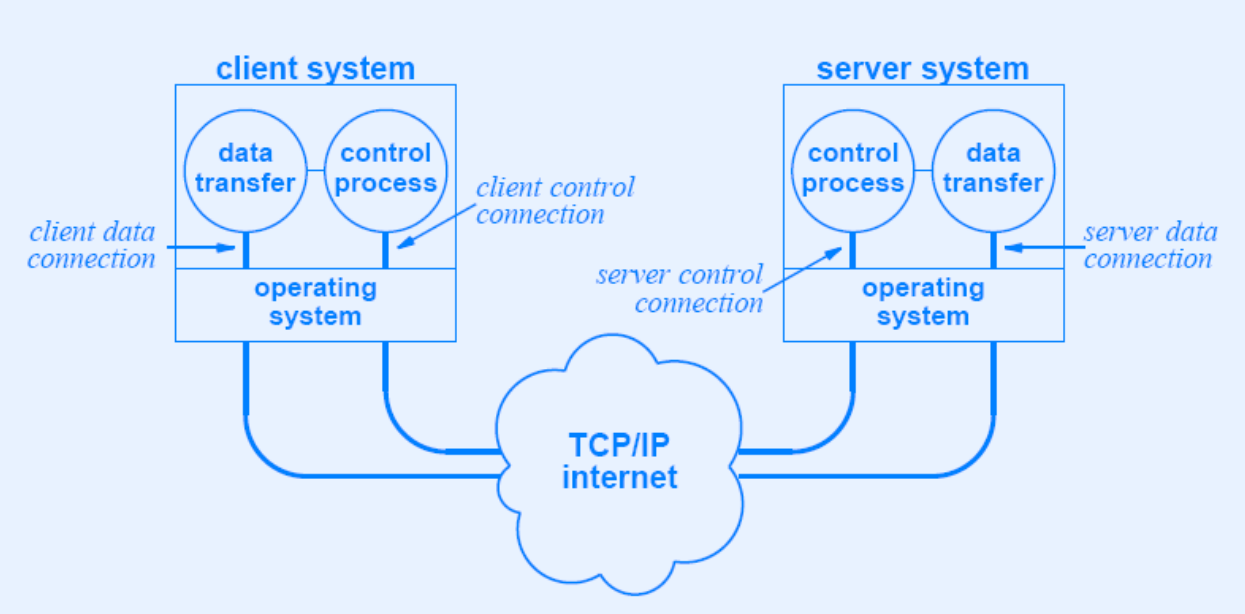
- Interactive access
  - FTP implementations provide interactive interfaces for human interactions
- Format specification (ASCII or EBCDIC)
  - Client can specify the type and format of stored data
- Authentication control (login and password)
  - The server refuses access to clients that do not provide valid credentials

FTP Process Model

- FTP server allows concurrent accesses
- Clients use TCP to connect to the server
- As for TELNET, a master server collects connections and dispatches them to slave servers
- Slave servers are different
  - They accept and handle the control connection (commands)
  - They use an additional process to control a data transfer connection (data)

FTP Process Model

- Separate processes handle
  - Interaction with users
  - Individual transfer requests



FTP's Use of TCP Connections

- *Data transfer connections and the data transfer processes that use them can be created dynamically when needed, but the control connection persists throughout a session. Once the control connection disappears, the session is terminated and the software at both ends terminates all data transfer processes*

Control Connection Vs. Data Connection

- Client
  - Creates process to handle data transfer
  - Allocates port and sends number to server over control connection
  - Process waits for contact
- Server
  - Receives request
  - Creates process to handle data transfer
  - Process contacts client-side

TCP Port Number

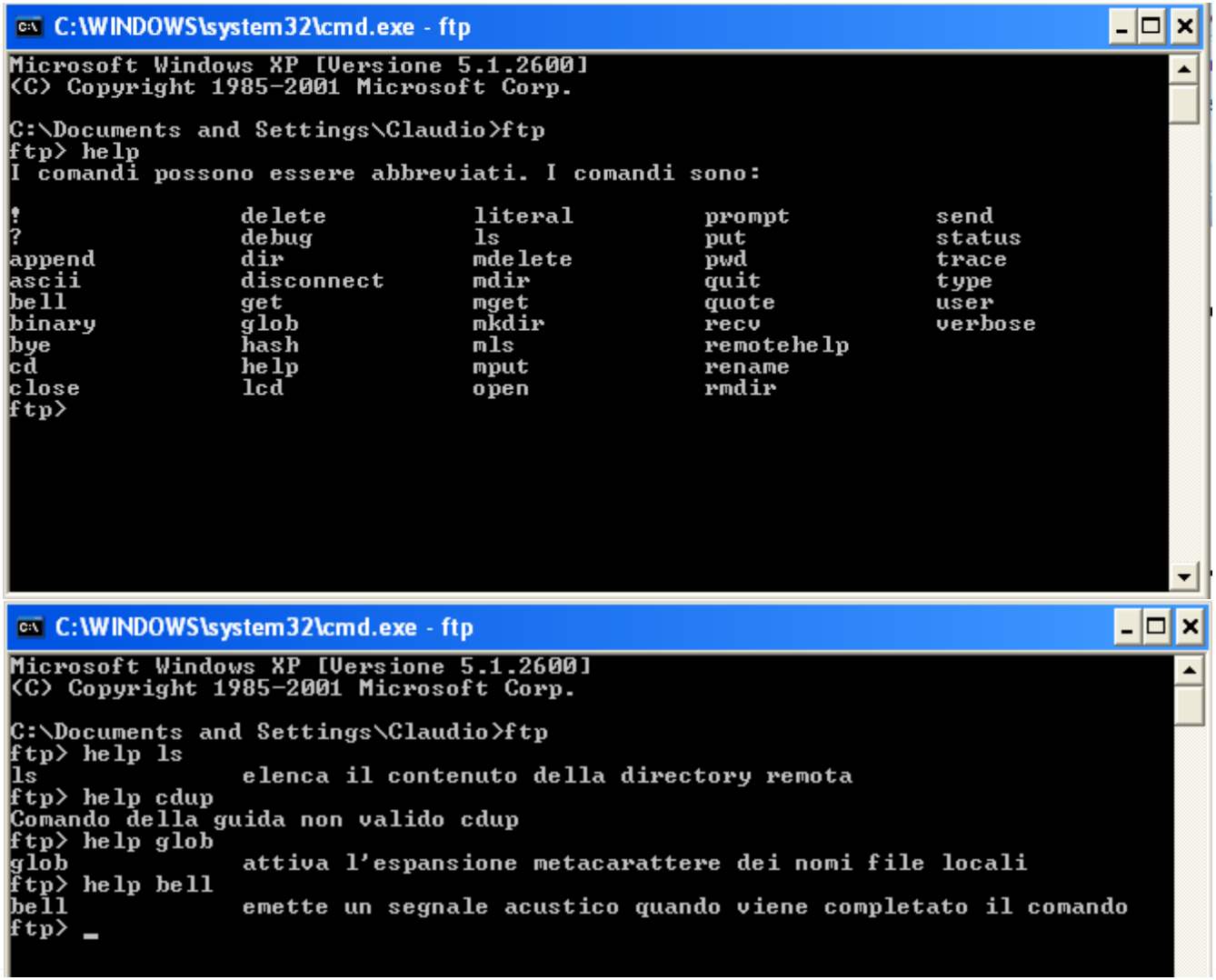
- *In addition to passing user commands to the server, FTP uses the control connection to allow client and server control processes to coordinate their use of dynamically assigned TCP protocol ports and the creation of data transfer processes that use those ports*

- *One connection for each data transfer*

FTP

- FTP uses the TELNET network virtual terminal for data passing along the control connection
- FTP does not allow option negotiation
- Management of FTP control connection much simpler than TELNET connection

Some Screenshot



Interactive Use of FTP

- Initially a command-line interface
  - User invokes client and specifies remote server
  - User logs in and enters password
  - User issues series of requests
  - User closes connection
- Currently
  - Most FTP initiated through browser
  - User enters URL or clicks on link
  - Browser uses FTP to contact remote server and obtain list of files
  - User selects file for download

Anonymous FTP

- Login *anonymous*
- Password *guest*
- Used for "open" FTP site (where all files are publicly available)
- Typically used by browsers

Secure File Transfer Protocols

- Secure Sockets Layer FTP (SSL-FTP)
  - Uses secure sockets layer technology
  - All transfers are confidential
- Secure File Transfer Program (sftp)
  - Almost nothing in common with FTP
  - Uses ssh tunnel
- Secure Copy (scp)
  - Derivative of Unix *remote copy (rcp)*
  - Uses ssh tunnel

Alternatives to FTP

- FTP complex and difficult
- Many applications do not need all FTP functionalities
- For instance, manage multiple TCP connection is difficult both for the client and for the server

Trivial File Transfer Protocol (TFTP)

- Whole-file copying
- Not as much functionality as FTP
- Code is much smaller (ROM memory)
- Intended for use on Local Area Network
- Diskless machine can use TFTP to obtain image at bootstrap

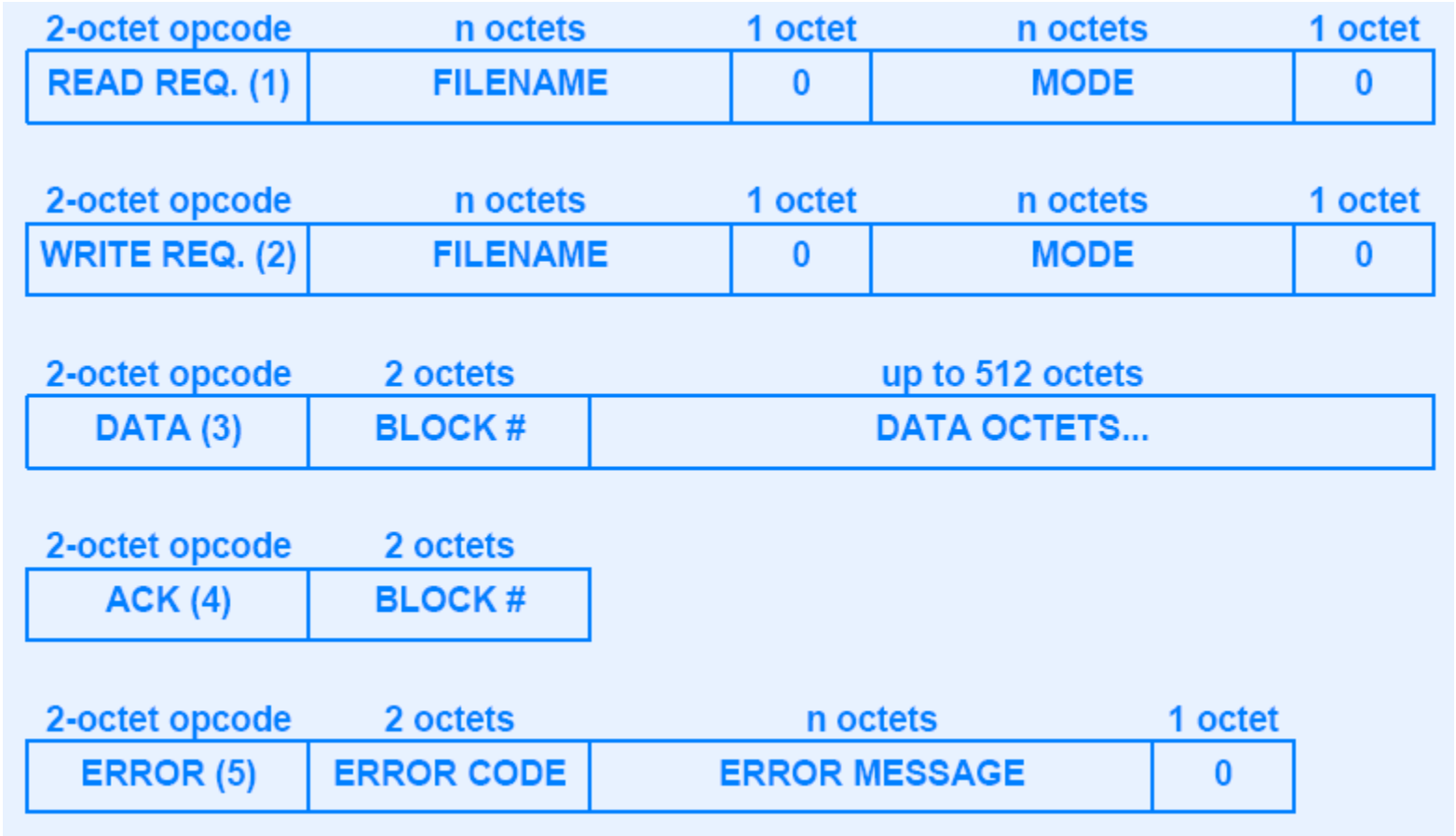
Trivial File Transfer Protocol (TFTP)

- Inexpensive and unsophisticated
- For applications that do not need complex interactions
  - Simple file transfer, no authentication
- Runs over UDP using timeout and retransmission
  - Sending-side sends 512 bytes (fixed size) and waits for ack from receiver-side

Trivial File Transfer Protocol (TFTP)

- TFTP flow
  - The first packet requests a transfer
    - File name, transfer mode (read/write)
  - File is read/write
    - Blocks are numbered starting from 1
    - Each block contains a number
    - The ack contains the number of the block
  - A block of less than 512 bytes identifies the EOF
  - An error message terminates the communication

TFTP Packet Types



TFTP Retransmission

- Symmetric (both sides implement timeout and retransmission)
  - Robust but high retransmission rate
- If the side sending data times out, it retransmits the data
- If the side responsible for ACK times out, it retransmits the acknowledgement

Sorcerer's Apprentice Bug

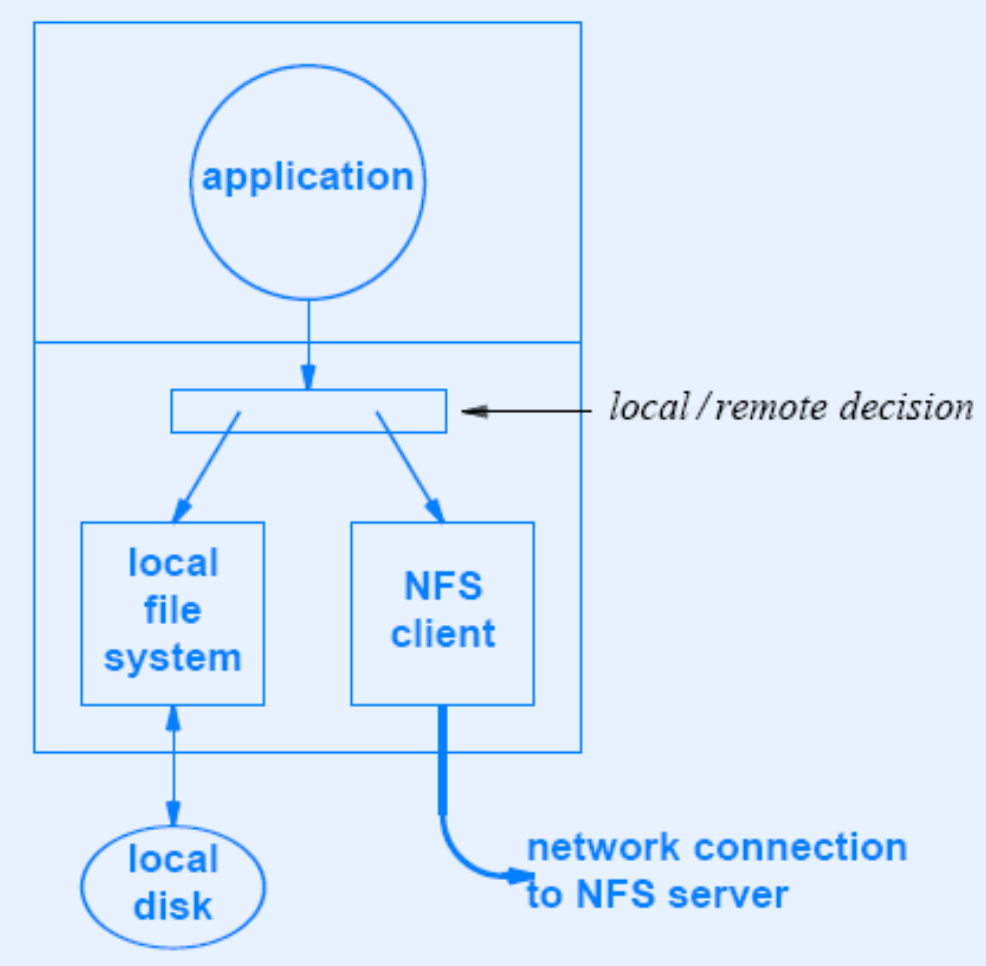
- Consequence of symmetric retransmission
- Duplicate packet is perceived as a second request, which generates another transmission
- Duplicate response triggers duplicate packets from the other end
- Cycle continues
- Message sent twice

Network File System (NFS)

- Protocol for online shared file access, not copying
- Developed by Sun Microsystems, now part of TCP/IP standards
- Transparent (application cannot tell that file is remote)

NFS Implementation

- Simulate a call to a local procedure
  - All details of the connection are hidded in the RPC package
  - The application calls the operation on a file without knowing whether it is local or remote



NFS Structure

- NFS protocol has three components
  - NFS protocol
  - RPC (Remote Procedure Call)
  - XDR (eXternal Data Rappresentation)

Remote Procedure Call (RPC)

- Also developed by Sun Microsystems, now part of TCP/IP standards
- Used in implementation of NFS
- Implement a paradigm for distributed client-server programming (we will see more on this)
- Relies on *eXternal Data Representation (XDR) standard for* conversion of data items between heterogeneous computers

eXternal Data Representation (XDR)

- A standard to represent the values transmitted in the messages exchanged by RPC programs
  - Similar to NVT
- XDR defines different types of data and how they must be encoded
  - Integers, unsigned integers, floating point numbers, ecc.
- XDR permits to describe complex data formats in a concise and unambiguous way

Summary

- Two paradigms for remote file sharing
  - Piecewise file access (online shared file access)
  - Whole file copying
- File Transfer Protocol (FTP)
  - Standard protocol for file copying
  - Separate TCP connection for each data transfer
  - Client and server roles reversed for data connection
- Examples of secure alternatives to FTP
  - SSL-FTP, sftp, and scp

Summary

- Trivial File Transfer Protocol (TFTP)
  - Alternative to FTP that uses UDP
  - Symmetric retransmission scheme
  - Packet duplication can result in Sorcerer's Apprentice problem
- Network File System (NFS)
  - Standard protocol for piecewise file access
  - Uses RPC and XDR

FTP

Approfondimenti

File Transfer Protocol (FTP)

- Most important application for file transfer
  - Originally available as a client/server application for ARPANET
- Runs over TCP
- Allows both file transfer and online shared access
- Based on authentication (username and password)

File Transfer Protocol (FTP)

- Client/server application
  - Server executes a FTP server
  - Wait for requests on a well-known port
  - Anonymous login provided (anonymous/guest)

- FTP server composed by
  1. Control process interacts with the remote control process
     - They exchange commands/responses and information on the ports
  2. Data transfer process transfers the real data

File Transfer Protocol (FTP)

- Client-side
  - The control process connects with the control process on the server side
    - ftp media.dti.unimi.it
  - A data transfer process is activated on a local port

- The set of commands used by FTP are a subset of the Telnet NVT (Network Virtual Terminal) and are encoded in ASCII characters

Example FTP

- 1xx = OK, lo farò
- 2xx = OK, fatto
- 3xx = OK, finora
- 4xx = NO, temporaneamente
- 5xx = azione richiesta
- Three digit code simplify the parsing (220)

```
% ftp pollo.crema.unimi.it
Connected to pollo.crema.unimi.it
220 pollo.crema.unimi.it FTP server (Version 6.8) ready.
Name (pollo.crema.unimi.it user_A) : anonymous
331 Guest login ok, send e-mail address as password.
Password: guest <------ (in realtà non visualizzata)
230 Guest login ok, access restrictions apply.
ftp> get utenti/studenti/tcpbook.tar bookfile
200 PORT command okay.
150 Opening ASCII mode data connection for tcpbook.tar (9895469 bytes).
226 Transfer complete.
9895469 bytes received in 22.76 seconds (4.3e+02 Kbites/s)
ftp> close
221 Goodbye.
```

Control Connection

- FTP uses the same approach as TELNET to communicate across the control connection
  - NVT ASCII character set
- Each command or response is only one short line terminated by End-of-Line token (two characters: CR+LF)

Data Connection

- The client must define the file type, the data structure, and the transmission mode

- File type
  - ASCII (default) – characters encoded in NVT ASCII
  - EBCDIC – the file transferred in EBCDIC
  - Binary – continuous stream of bits (no encoding)

Data Connection

- The client must define the file type, the data structure, and the transmission mode

- Data structure
  - File structure (D) – File is a continuous byte stream
  - Record structure – File is an array of records (C-structures)
  - Page structure – File consists of pages, each page has a page number and page header. Pages can be accessed randomly

Data Connection

- The client must define the file type, the data structure, and the transmission mode

- Transmission mode
  - Stream mode (D) – File is a continuous stream of bytes directly delivered to TCP
    - TCP is responsible for breaking down the data to be transmitted
  - Block mode – Data delivered to TCP in blocks. Each block has a 3-byte header:
    - <block descriptor, size of block>

Data Connection

- The client must define the file type, the data structure, and the transmission mode

- Transmission mode
  - Compressed mode – For big files data can be compressed
    - Use run-length encoding

Data Transfers - *Active (PORT) and Passive (PASV)*

- The client program can specify *active mode by sending the "PORT" command to instruct that the server should connect back to a specified IP address and port number and then send the data, or*

- A client program can choose *passive mode by using the "PASV" command to ask that the server tell the client an IP address and port number that the client can connect to and receive the data*

Data Transfers - *Active (PORT) and Passive (PASV)*

- PORT is used to have the server connect to the client
- PASV is used to have the client connect to the server
- Since the client connects to the server to establish the control connection, it would seem logical that the client should connect to the server to establish the data connection, which would imply that PASV would be preferred (and at the same time eliminate the single biggest problem with FTP and firewalls)
- **Mysteriously**, the implementers chose to specify in the FTP specification that PORT should be preferred and PASV need not be implemented at all by FTP client programs

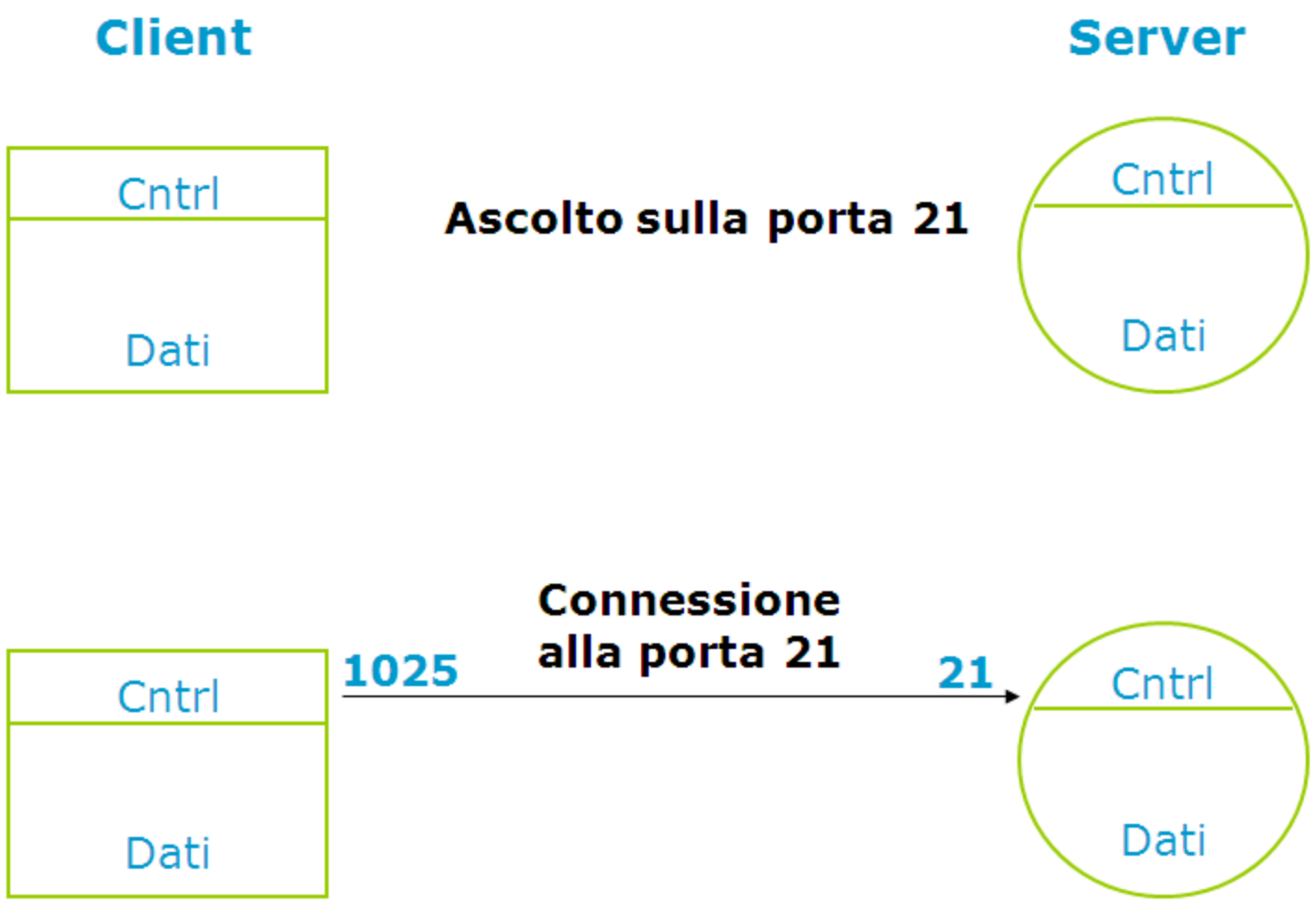
Active FTP

- Client -> Server (control connection)
  - Client port random
  - Server port 21
- Server -> Client (data transfer connection)
  - Client port random (used for TCP connection with the server's process)
  - Server port 20 (reserved for FTP data transfer)
  - Server-side does not accept connection from arbitrary process
  - The server uses the TCP active open request to specify the client port and its local port

Active FTP

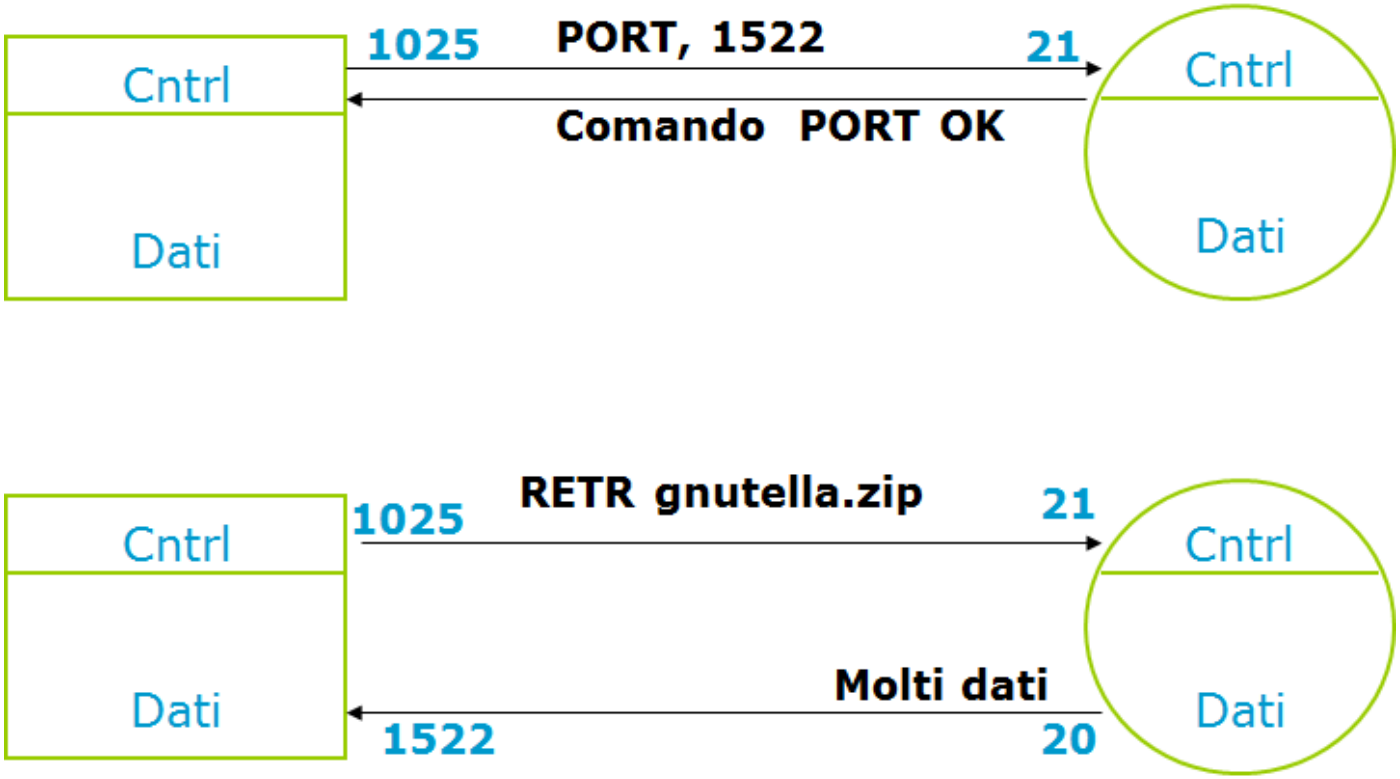
- When a file transfer is request, the server connects to the client port for data transfer
  - The server uses a well-known port (20)
  - Port number communicated to the server via the control process

Active FTP


Client
Server
Cntrl
Dati
Ascolto sulla porta 21
Cntrl
Dati
Connessione
alla porta 21
1025
21
Cntrl
Dati
Cntrl
Dati

Active FTP

- The client never interacts with a random port of an untrusted server



Example Sessions Using Active Data Transfers

- Client: USER anonymous
- Server: 331 Guest login ok, send your e-mail address as password
- Client: PASS NcFTP@
- Server: 230 Logged in anonymously
- Client: PORT 192,168,1,2,7,138 The client wants the server to send to port number 1930=7*256+138 on IP address 192.168.1.2.
- Server: 200 PORT command successful.
- Client: LIST
- Server: 150 Opening ASCII mode data connection for /bin/ls. The server now connects out from port 21 to port 1930 on 192.168.1.2.
- …
- Server: 226 Listing completed. That succeeded, so the data is now sent over the established data connection.
- Client: QUIT
- Server: 221 Goodbye.

Firewall problems with FTP

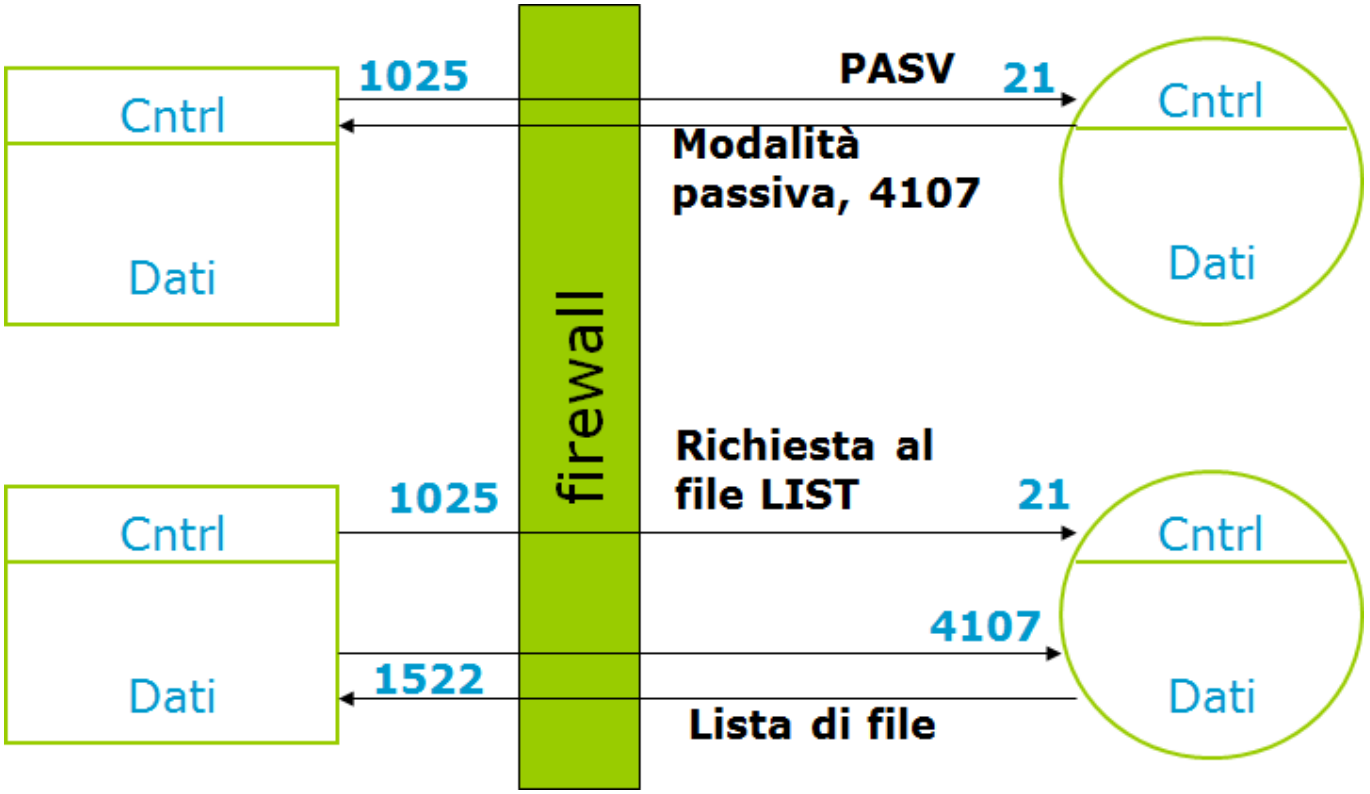
- Client-side Firewalls
  - The client is behind a firewall and cannot be reached directly from the outside
- NATs
  - The client is behind a NAT device and cannot be reached from the outside

Client-side Firewalls: Solution 1

- Solution 1: The client user should configure their FTP client program to use PASV rather than PORT

Passive FTP

- The server cannot initiate a connection to the client (e.g., firewall)



Example Sessions Using Passive Data Transfers

- Client: USER anonymous
- Server: 331 Guest login ok, send your e-mail address as password.
- Client: PASS NcFTP@
- Server: 230 Logged in anonymously.
- Client: PASV The client is asking where he should connect.
- Server: 227 Entering Passive Mode (172,16,3,4,204,173) The server replies with port 52397 on IP address 172.16.3.4.
- Client: LIST
- Server: 150 Data connection accepted from the client; transfer starting. The client has now connected to the server at port 52397 on IP address 172.16.3.4.
- …
- Server: 226 Listing completed. That succeeded, so the data is now sent over the established data connection.
- Client: QUIT
- Server: 221 Goodbye.

Client-side Firewalls: Solution 2

- Using passive mode may not solve the problem if there is a similar restrictive firewall on the server side
- Solution 2: A better solution is for the network administrator of the client network to use high-quality Network Address Translation (NAT) software
  - Devices can keep track of FTP data connections, and when a client on a private network uses "PORT" with an internal network address, the device should dynamically rewrite the packet containing the PORT and IP address and change the address so that it refers to the external IP address of the routing device
  - The device would then have to route the connection incoming from the remote FTP server back to the internal network address of the client
- From the example above we had:
  - Client: PORT 192,168,1,2,7,138

Client-side Firewalls: Solution 2

- When the packet containing this PORT reaches the routing device, it should be rewritten like this, assuming the external address is 17.254.10.26
- Client: PORT 17,254,10,26,7,138
- The remote server would then attempt to connect to 17.254.10.26:1930
  - 1930=7*256+138
- The routing device in this example would then forward all traffic for this connection to and from the client address at 192.168.1.2:1930

More Information

- http://www.ncftp.com/ncftpd/doc/misc/ftp_and_firewalls.html

Esercizio

- Esercizio 2 - Tema d’esame 10 Febbraio 2010

Esercizio

- Esercizio 2 - Tema d'esame 7 Settembre 2010